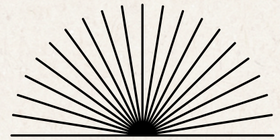
AYUDANTÍA 6 ARQUITECTURA DE SOFTWARE

Disponibilidad, Performance, Escalabilidad y Seguridad

Benjamín Aceituno - Arquitectura de Software



Escenario de QA y tácticas

Un atributo de calidad no se puede probar si se escribe como “el sistema debe ser rápido” o “seguro”.

Para volverlo verificable se escribe como un escenario de 6 partes, y es el mismo método para los cuatro atributos que se verán hoy.

- **Source:** Quién o qué dispara el evento.
- **Stimulus:** El evento que llega (peak de carga, ataque, caída).
- **Artifact:** La parte del sistema afectada.
- **Environment:** En qué condición está el sistema (normal, peak, degradado).
- **Response:** Qué hace el sistema.
- **Response measure:** Cómo se mide la respuesta, con un número testeable.

Ejemplo: En una app de delivery de comida, 3.000 usuarios (source) reservan a la vez (stimulus) en el módulo de búsqueda de restaurantes(artifact) un viernes a las 18:00 (environment). El sistema muestra restaurantes disponibles y su menú (response) con un p99 menor a 800 ms (response measure).

Disponibilidad

La disponibilidad es la probabilidad de que el sistema esté operativo cuando se necesita. Se puede explicar con tres conceptos: Un fault (la causa, como un disco que falla) provoca un error (estado interno incorrecto), que puede terminar en un failure (lo que el usuario ve).

La resiliencia consiste en que el fault nunca llegue a ser failure.

Disponibilidad = MTBF / (MTBF + MTTR)

MTBF: Tiempo medio entre fallas.

MTTR: Tiempo medio para volver a servicio.

RTO: El tiempo máximo que puedo estar caído en una falla.
RPO: La cantidad máxima de datos que puedo perder.

Cada “nueve” extra equivale a unas 10 veces menos downtime, pero exige más redundancia y costo: 99 % son ~3 días al año, 99,9 % son ~8 h 46 min, 99,99 % son ~52 min y 99,999 % son ~5 min.

Ejemplo: Con MTBF de 1.000 h y MTTR de 1 h, la disponibilidad es 99,9 %. Si el MTTR baja a 6 minutos, sube a 99,99 %, sin tocar el MTBF.

Tácticas de disponibilidad y patrones de resiliencia

Las tácticas se agrupan en tres: detectar, recuperar y prevenir.

Detectar: Ping/echo (el monitor pregunta “¿estás vivo?”), heartbeat (el componente avisa “sigo vivo”) y timeout.
Recuperar: Redundancia (activa, pasiva o spare), rollback, retry y degradation.
Prevenir: Removal from service y transactions.

En sistemas distribuidos estas tácticas aparecen con nombres modernos:

Timeout: Nunca esperar indefinidamente a un servicio remoto. Se acota la espera y se falla rápido.
Retry con backoff + jitter: Reintentar espaciando los intentos, solo para fallas transitorias y con tope. Exige idempotencia.
Circuit breaker: Pasa de closed a open tras N fallos, y luego a half-open para probar si el servicio volvió. Corta la cascada de fallas.
Bulkhead (mamparo): Pools o colas separados por dependencia, para que un fallo no hunda a todo el sistema.

Ejemplo: Una app de delivery de farmacia depende de un servicio externo que verifica recetas. Si se cae, el breaker se abre y la app deja esos pedidos “en revisión” mientras el resto sigue funcionando.

Performance

La performance mide qué tan bien responde el sistema en el tiempo.

Latencia: Tiempo entre el estímulo y la respuesta. Es lo que siente el usuario.
Throughput: Trabajo por unidad de tiempo (req/s).
Percentiles: El promedio esconde las respuestas lentas, por eso los SLO se escriben en p95 o p99. Un p99 de 500 ms significa que el 99 % de las peticiones responde en menos de 500 ms.
Ley de Little: $L = \lambda \cdot W$ (requests en el sistema = tasa de llegada × tiempo de permanencia).

Ejemplo: Una API de una app de transporte recibe 2.000 req/s y cada una tarda 0,25 s. Hay 2.000 × 0,25 = 500 requests en vuelo, y con ese número se dimensionan hilos y conexiones.

Tácticas de performance y escalabilidad

Las tácticas de performance se dividen en dos grupos.

Controlar la demanda: Rate limiting (se encola lo que sobra), priorizar eventos (el pago antes que las recomendaciones) y reducir el overhead.
Gestionar los recursos: Más recursos, concurrencia, copias de cómputo (réplicas con balanceador) y copias de datos (caching y réplicas de lectura).

Escalabilidad: Es la capacidad del sistema de seguir cumpliendo su escenario cuando la carga se multiplica. La performance responde a tiempo hoy, y la escalabilidad asegura que se siga respondiendo a tiempo con 10 veces más usuarios.

Para escalar hay dos caminos:
Scale up: Un nodo más grande. Es simple, pero tiene un techo y sigue siendo un punto único de falla.
Scale out: Más nodos tras un balanceador. Es elástico, pero exige nodos stateless: la sesión se guarda en un almacén compartido (como Redis) y no en el nodo.
Caching: Se aplica por capas (cliente, CDN, aplicación y réplicas de BD). Lo difícil es invalidarlo: se usan TTL, cache-aside y write-through.
Sharding: Reparte los datos entre varias bases según una clave, y así reparte las escrituras.

Ejemplo: Una tienda de ropa llega al límite de su servidor. Con scale up compra uno más grande, y queda con un solo servidor. Con scale out agrega dos más con un balanceador: crece sin techo y gana disponibilidad.

Seguridad

La seguridad protege los datos y servicios del acceso no autorizado, sin dejar de atender al usuario legítimo.

Se apoya en la tríada CIA, que es lo que un ataque busca romper:

Confidencialidad: Que solo vea los datos quien corresponde.

Integridad: Que los datos no se modifiquen sin autorización.

Disponibilidad: Que el servicio siga funcionando.

Se sostienen con la autenticación (quién eres), la autorización (qué puedes hacer) y el no repudio (no puedes negar lo que hiciste).

Ejemplo: En el sistema de notas de una universidad, alguien intenta modificar una nota sin permiso. El sistema bloquea la acción, registra el intento y avisa al administrador en menos de 1 minuto.

Tácticas de seguridad y diseño moderno

Las tácticas de seguridad se agrupan en cuatro familias:

Detectar: Intrusion detection, service denial, integrity y delay.
Resistir: Autenticar (MFA, certificados), autorizar con privilegio mínimo (RBAC / ABAC), limitar el acceso y la exposición, cifrar y cambiar los valores por defecto.
Reaccionar: Revoke access, lock computer tras intentos fallidos e inform actors.
Recuperar: Restore, apoyado en las tácticas de disponibilidad. El audit trail sirve para el análisis forense y el no repudio.

Además, hay tres ideas de diseño moderno:

Defensa en profundidad: Capas independientes de protección (WAF, gateway, autenticación, cifrado, auditoría). Si una cae, las otras siguen.
Zero trust: No existe una red confiable por defecto. Cada request se autentica y autoriza, aunque venga de la red interna.
STRIDE: Clasifica las amenazas, y cada una apunta a una propiedad que se quiere proteger:

Tácticas de seguridad y diseño moderno

Amenaza	Propiedad
Spoofing	Autenticación
Tampering	Integridad
Repudiation	No repudio
Information disclosure	Confidencialidad
Denial of service	Disponibilidad
Elevation of privilege	Autorización

Ejemplo: Un sistema de fichas clínicas bloquea la cuenta tras 5 intentos fallidos (lock computer), exige un segundo factor (MFA) y registra cada acceso (audit trail).

Trade-offs

Mejorar un atributo casi siempre cuesta algo en otro. El trade-off aparece al combinar tácticas, y se registra en un ADR como consecuencia (-).

Seguridad ↔ performance: TLS en cada salto, cifrado y validaciones suman latencia y CPU.

Disponibilidad ↔ consistencia y costo: Las réplicas cuestan, y ante una partición hay que elegir entre consistencia o disponibilidad (CAP).

Resiliencia ↔ carga: Los retries sin backoff ni breaker multiplican el tráfico sobre el servicio caído.

Performance ↔ consistencia: El caché puede servir datos viejos (precio, stock, saldo).

Escalabilidad ↔ seguridad: Más nodos significan más superficie de ataque y más secretos que gestionar.

Ejemplos:

- “Cifrar toda la comunicación entre servicios” → (Gana seguridad, pierde performance)
- “Agregar un caché para el catálogo” → (Gana performance, pierde consistencia)
- “Agregar 10 réplicas” → (Gana disponibilidad, pierde costo y simplicidad)

Ejercicio

“MaicolPay” es una app para pagar cuentas de servicios (agua, luz, gas). El día 10 de cada mes, que es el peak de vencimientos, la app recibe unas 5.000 req/s. Para cobrar depende de un adquirente externo (Transbank). Si este se cae, hoy la app queda con las peticiones esperando y se cae completa. La empresa exige que nunca se cobre dos veces una misma cuenta.

Redacte un escenario de disponibilidad con sus 6 partes y elija 2 tácticas justificando el trade-off que acepta.

- Source:
- Stimulus:
- Artifact:
- Environment:
- Response:
- Response measure:

Táctica 1:

Táctica 2:

Trade-off:

Contacto

+569 86031881

benjamin.aceituno@mail.udp.cl

benja.mp4

https://github.com/benjamp4/Arquitectura-de-Software-2026-1